



Text-to-SQL Analytics Engine

Codebasics AI Engineering Bootcamp • Assignment 3 • Capstone Project

Vijay Mamilla | vijay.mamilla@gmail.com

github.com/vijaymamilla/querymind

Problem Statement

Why does QueryMind exist?

The Problem

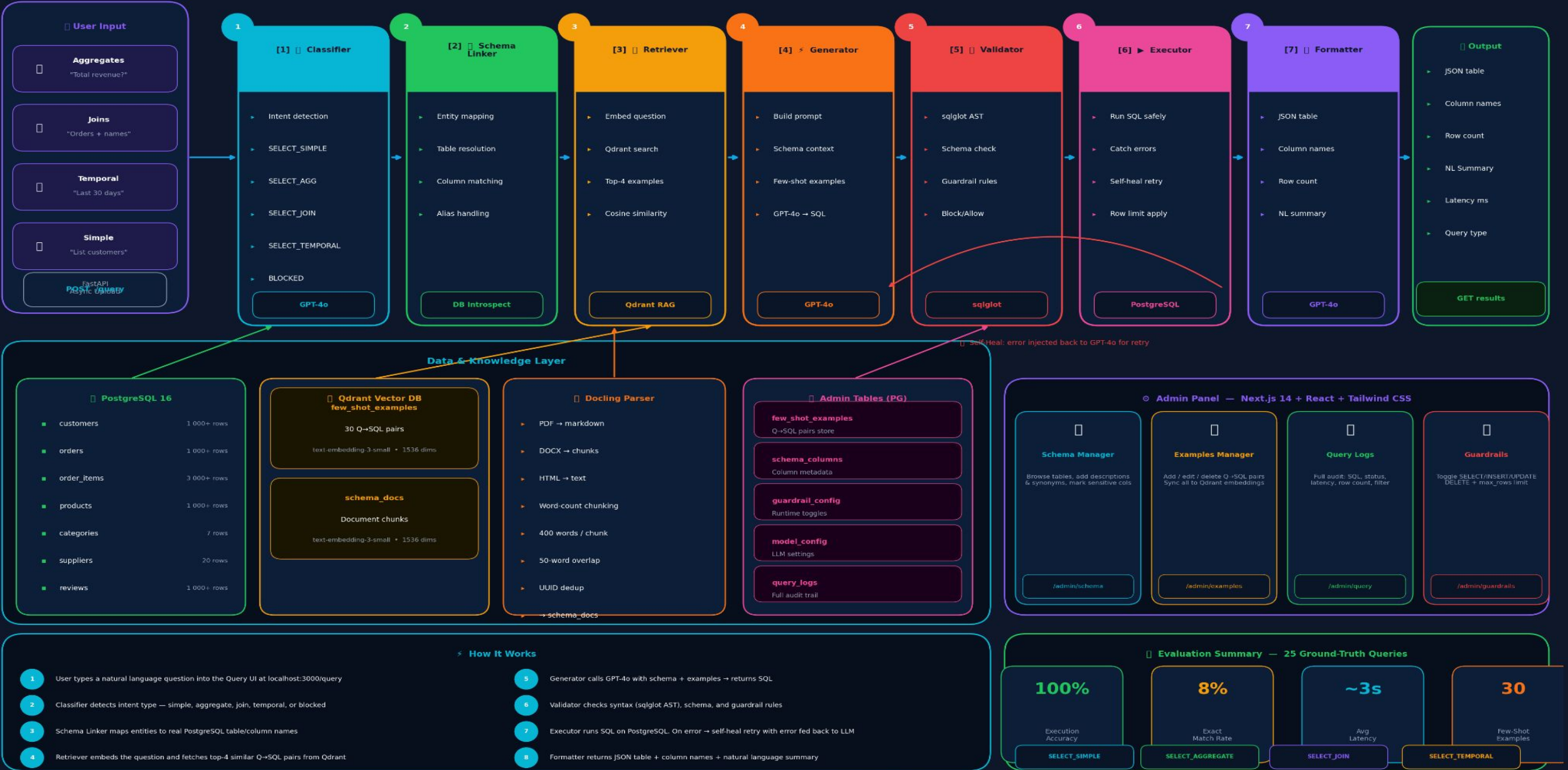
- ▶ Business analysts need data insights but can't write SQL
- ▶ Data teams are bottlenecked by ad-hoc query requests
- ▶ Traditional BI tools require pre-built dashboards — no flexibility
- ▶ Natural language interfaces exist but lack guardrails and auditability

QueryMind Solves It

- ▶ Type a question in plain English → get SQL + results instantly
- ▶ Configurable guardrails prevent dangerous queries (DELETE, DROP)
- ▶ Every query logged with generated SQL, latency, and status
- ▶ Admin panel to tune behavior without touching code

QueryMind AI — Text-to-SQL Analytics Engine

OpenAI GPT-4o Qdrant Docling PostgreSQL 16 FastAPI Next.js 14 sqlglot Docker



Technology Stack

Modern async Python backend + React frontend

LLM & Embeddings	OpenAI GPT-4o (SQL generation) • text-embedding-3-small (1536-dim, cosine)
Vector Database	Qdrant — two collections: few_shot_examples & schema_docs
Document Parsing	Docling — PDF, DOCX, HTML → chunked markdown → Qdrant index
Backend	FastAPI (async) • SQLAlchemy async + asyncpg • pydantic-settings
Relational DB	PostgreSQL 16 — e-commerce domain, 7 tables, 1 000+ rows each
SQL Tooling	sqlglot v30 — syntax validation, AST-based table extraction
Frontend	Next.js 14 • TypeScript • Tailwind CSS
Infrastructure	Docker Compose (Postgres + Qdrant) • uv package manager

Domain & Database Schema

E-commerce PostgreSQL database — synthetic data

customers

id, first_name, last_name, email, country, joined_at

orders

id, customer_id, status, total_amount, created_at

order_items

id, order_id, product_id, quantity, unit_price

products

id, name, category_id, supplier_id, unit_price, stock_quantity

categories

id, name, description

suppliers

id, name, email, country

reviews

id, product_id, customer_id, rating, body, created_at

Key Features

What makes QueryMind production-ready

Few-Shot RAG

Top-4 semantically similar Q→SQL examples retrieved from Qdrant at query time. 30 curated examples covering 4 query types.

Self-Healing SQL

On execution error, the SQL + error message is injected back into GPT-4o for one automatic retry. Transparent in query logs.

Document Ingestion

Upload PDF/DOCX/HTML schema docs via Docling. Chunks indexed to Qdrant schema_docs collection for context-aware generation.

Guardrails

Runtime toggles for SELECT / INSERT / UPDATE / DELETE. Max-rows limit (default 500). Validated before execution — never bypassed.

Full Audit Log

Every query logged: NL question, generated SQL, status, row count, latency. Filterable by success / error / blocked.

Runtime Config

Switch LLM model (gpt-4o ↔ gpt-4o-mini), SQL dialect, temperature — all via Admin UI without restarting the server.

Admin Panel — 5 Management Screens

Full React UI — no code changes needed



Schema Manager

- Browse live DB schema (7 tables)
- Add descriptions & synonyms per column
- Mark sensitive columns (masked in output)
- Refresh from live DB at any time



Examples Manager

- Add / edit / delete Q→SQL pairs
- Filter by query type
- Sync all to Qdrant embeddings
- 30 examples pre-seeded across 4 types



Query Logs

- Full audit trail of every query
- Filter: All / Success / Error / Blocked
- Expand row to see generated SQL
- Pagination — 20 per page



Guardrails Config

- Toggle SELECT/INSERT/UPDATE/DELETE
- Set max_rows limit
- Changes take effect immediately
- Blocked queries logged automatically



Model Config

- Switch GPT-4o / GPT-4o-mini
- Change SQL dialect
- Adjust temperature & max_tokens
- No server restart required

Evaluation Results

25 ground-truth queries • 4 query types • automated harness

100%

Execution Accuracy

25 / 25 queries ran
successfully

8%

Exact Match Rate

LLM adds LIMIT 500
guardrail — expected

~3s

Avg Latency

End-to-end: classify
→ generate → execute

Query Type	Count	Exec Accuracy	Exact Match
SELECT_SIMPLE	6	100%	16.7%
SELECT_AGGREGATE	7	100%	0.0%
SELECT_JOIN	7	100%	14.3%
SELECT_TEMPORAL	5	100%	0.0%

Note: Exact match is low because the pipeline appends LIMIT 500 (max_rows guardrail) to all queries. Results are correct — execution accuracy is the primary metric.

Live Demo — Sample Queries

Type these in the Query interface at localhost:3000/query

SIMPLE

"List all pending orders"

AGGREGATE

"How many customers do we have?"

AGGREGATE

"How many orders are there per status?"

JOIN

"List the top 5 customers by total amount spent"

TEMPORAL

"Show orders placed in the last 30 days"

SIMPLE

"Show products that are out of stock"

AGGREGATE

"What is the total revenue from delivered orders?"

JOIN

"Show all orders with customer names"

JOIN

"Which products have been ordered more than 10 times?"

TEMPORAL

"Show monthly order counts for this year"

Guardrails Demo

Runtime access control — no code changes

Step 1

Go to Admin → Guardrails (localhost:3000/admin/guardrails)

Show toggles: allow_select=ON, allow_insert=OFF, allow_update=OFF, allow_delete=OFF

Step 2

Go to Query and type: "Delete all customers"

Result: BLOCKED  — "Operation DELETE is not permitted by guardrails"

Step 3

Go back to Guardrails — turn OFF allow_select

Try any SELECT query — it is now blocked too

Step 4

Turn allow_select back ON

Queries work again immediately — no server restart

Project Structure & Modularity

Each pipeline stage is an independent, swappable module

```
querymind/
├── backend/app/
│   ├── pipeline/
│   │   ├── classifier.py      ← Stage 1: intent classification
│   │   ├── schema_linker.py  ← Stage 2: entity → schema mapping
│   │   ├── retriever.py      ← Stage 3: Qdrant few-shot retrieval
│   │   ├── ingester.py       ← Docling doc parser + Qdrant indexer
│   │   ├── generator.py      ← Stage 4: OpenAI SQL generation
│   │   ├── validator.py      ← Stage 5: guardrails + syntax check
│   │   ├── executor.py       ← Stage 6: safe execution + self-heal
│   │   └── formatter.py      ← Stage 7: JSON + NL summary
│   └── admin/routes.py       ← All CRUD admin API endpoints
├── frontend/src/app/admin/   ← React admin UI (5 screens)
├── scripts/
│   ├── seed_db.py           ← Synthetic e-commerce data
│   └── seed_examples.py     ← 30 few-shot examples
└── eval/
    ├── ground_truth.py      ← 25 Q+SQL ground-truth pairs
    └── run_eval.py          ← Automated evaluation harness
```

QueryMind AI

- ✓ 7-stage modular NL→SQL pipeline with self-healing
- ✓ Qdrant vector search — 30 few-shot examples across 4 query types
- ✓ Runtime guardrails — configurable without code changes
- ✓ Full audit logging — every query tracked
- ✓ React admin panel — 5 management screens
- ✓ Evaluation harness — 100% execution accuracy on 25 queries
- ✓ Docling document ingestion — schema docs → Qdrant context



GitHub

github.com/vijaymamilla/querymind



Backend API

localhost:8000/docs



Admin UI

localhost:3000